`blaris`: Windowless Random-Access LZSS for Constrained Microcontrollers

Ihor Lukianov
Faculty of Mathematics and Computer Science, Jagiellonian University, Kraków
ihor.lukianov@student.uj.edu.pl

Abstract

Embedded LZ77 decompressors (such as `heatshrink` and `tamp`) require an in-RAM sliding window that couples memory usage to compression ratio and incurs high latency on random-access reads. We present `blaris`, a lightweight, windowless LZSS decompressor for execute-in-place (XIP) random access on constrained microcontrollers. By partitioning tokens into bidirectionally traversable streams with power-of-two checkpoints, `blaris` resolves back-references directly in the compressed payload, achieving compression ratios comparable to high-window `heatshrink` and `tamp` configurations without allocating a RAM window. Targeting compress-on-host, decompress-on-device workflows, `blaris` assumes trusted build-time payloads and omits runtime error and bounds checks. On RV32IMC, it requires just 548–784 B of Flash and 104–112 B of total RAM, achieving orders-of-magnitude faster small-slice random reads than ratio-matched, block-indexed `heatshrink` and `tamp`.

Motivations

On resource-constrained microcontrollers with only a few kilobytes of Flash and RAM, every byte counts. Static assets such as UI bitmaps, fonts, device trees, and configuration tables often consume large amounts of Flash even though they are accessed infrequently.

Compressing these assets in Flash saves space, but general-purpose decompressors like Zstandard [3] require over 100 KiB of memory, and byte-aligned codecs like LZ4 require multi-kilobyte blocks and windows to achieve competitive ratios. Bit-level embedded alternatives such as `heatshrink` [4], `tamp` [5], and `uzlib` [6] use far less memory, but as LZ77 [1] stream decoders, they still require an in-RAM sliding window to resolve back-references. Because the window size directly determines both RAM usage and compression ratio, better compression always costs more RAM.

Sliding-window decoders are also inefficient for random access to small data slices. To read a few bytes from the middle of a compressed block, the decoder must allocate the full window and sequentially decompress all preceding data from the start of the block. Random-access parsings such as LZ-End [8, 9] avoid a sliding window by forcing matches to end at prior phrase boundaries, but are designed for large, repetitive text indexes [10] using auxiliary phrase tables and rank/select bitvectors rather than kilobyte-scale MCU budgets.

This paper introduces `blaris`, a windowless variant of LZSS [2] that removes the sliding-window buffer entirely. By organizing the compressed data into streams that can be traversed both forward and backward, `blaris` uses a small, fixed amount of RAM regardless of the lookback range. We further optimize random-access reads by adding compact state checkpoints, host-side optimal parsing, and reference-chain flattening.

Windowless LZSS

Standard LZSS compresses data into a sequence of tokens, where each token is either a raw literal run or a back-reference match (d, m) that copies m bytes from a distance of d bytes earlier. Conventional decoders keep recently decompressed bytes in a RAM sliding window so that matches can be copied immediately.

`blaris` eliminates the RAM window by resolving each byte directly within the compressed payload. To decode a byte at an uncompressed offset X, the decoder finds the token covering X:

1. If the token is a **literal run**, the byte is read directly from the `literals` stream and emitted.
2. If the token is a **match** with distance d, the target byte is identical to the byte at offset $X - d$. The decoder updates $X \leftarrow X - d$ and walks backward through the compressed tokens to find the token covering the new offset.

This process repeats, following the chain of matches backward until it lands on a literal run. In practice, rather than resolving one byte at a time, the decoder tracks the remaining contiguous span w (initialized to the requested slice length) and narrows $w \leftarrow \min(w, P + \text{length} - X)$ at each hop, copying all w resolved bytes from the final literal run at once. While walking token chains takes more CPU cycles than copying from a RAM buffer, it completely removes the sliding window.

To make walking forward and backward across tokens fast and deterministic, `blaris` splits the compressed payload

into three separate streams:

- **controls** (1 byte per token): values 0..15 encode a literal run of 1..16 bytes; values 16..255 encode a match, packing all 240 combinations of match length (2..16 bytes) and distance bit-width (1..16 bits).
- **literals** (1 byte per literal): stores the raw literal bytes contiguously.
- **distance_bits** (bit-packed stream): stores the variable-width distance bits for each match token.

Because every control token is exactly 1 byte and specifies the exact number of literal bytes or distance bits it consumes, the decoder can step both forward and backward across all three streams in lockstep. The decoder tracks a 4-tuple state (P, C, L, D), representing the uncompressed byte position P, the `controls` index C, the `literals` index L, and the bit offset D in `distance_bits`:

- **Step forward**: read `controls[`C`]`. For a literal run of length k, add k to P and L. For a match of length m with b distance bits, add m to P and b to D. Increment C by 1.
- **Step backward**: decrement C by 1, read `controls[`C`]`, and subtract the corresponding k or (m, b) values from P, L, and D.

With up to 16 distance bits per match, `blaris` addresses a 64 KiB lookback range. To reach 128 KiB or 256 KiB, match distances can optionally be aligned to powers of two, trading a small loss in unaligned matches for a wider lookback reach.

Optimizations

I Checkpoints

Walking backward token by token uses constant $\mathcal{O}(1)$ RAM, but stepping across thousands of tokens for a single back-reference is slow. To keep lookups fast, we add *checkpoints*: precomputed snapshots of the decoder state (P, C, L, D) stored in the payload header at compression time.

Storing four raw 32-bit integers per checkpoint would cost 16 bytes each, hurting the compression ratio on small files. To shrink this overhead, we place checkpoints along a regular power-of-two grid so that the i-th checkpoint starts at the first token at or after grid boundary $i \cdot 2^{\text{SPREAD_LOG}}$:

$$i \cdot 2^{\text{SPREAD_LOG}} \le P_i < i \cdot 2^{\text{SPREAD_LOG}} + 16. \quad (1)$$

Because the maximum token length is 16 bytes, position P_i is always within 0..15 bytes of the grid boundary. Its upper bits are already known from the checkpoint index i, so we only need to store a 4-bit remainder for P_i. Similarly, we split the bit offset D into a byte index ($D \gg 3$) and a 3-bit remainder ($D \,\&\, 7$). Packing the 4-bit P remainder and 3-bit D remainder into one metadata byte uses just 7 bits. Depending on the chosen offset width (2, 3, or 4 bytes per stream index), each checkpoint takes only 7, 10, or 13 bytes. To prevent unaligned memory reads on strict architectures, checkpoints are split into two parallel arrays:

- **metas** (1 byte per checkpoint): packs the 4-bit P remainder and 3-bit D bit-offset.
- **offsets** ($3 \times \{2, 3, 4\}$ bytes per checkpoint): stores the byte-aligned $(C, L, D \gg 3)$ indices.

Because checkpoints sit on a power-of-two grid, the decoder can find the checkpoint nearest to any target offset X in $\mathcal{O}(1)$ time using a single bit-shift ($X \gg$ SPREAD_LOG). If the checkpoint's +0..15 byte remainder places it slightly past X, the decoder simply steps backward by one token. When following a match to $X' = X - d$, the decoder jumps to the nearest checkpoint whenever doing so saves at least 32 bytes of linear token walking, reducing each hop to a few token steps.

II Optimal Parser & Chain Flattening

`blaris` treats compression ratio as its primary goal. Because compression runs offline on the host, encoding speed does not matter. We therefore adapt the dynamic programming optimal parser from `lzmpo` [11] (similar to shortest-path LZ parsing in LZMA [12]) to find the exact sequence of literal runs and matches with the smallest total bit cost.

The parser first builds a single-byte hash chain linking each position i to preceding positions $j < i$ that share the same byte value. It then scans forward from $i = 0$ to N. At each position i, it walks the hash chain across the 64–256 KiB lookback range to populate a local table `sheet`[2..16], where `sheet`[k] stores the cheapest valid match distance for an exact match of length k. Next, a downward pass from $k = 15$ down to 2 replaces `sheet`[k] with `sheet`[k+1] whenever a longer match is cheaper in bits than a shorter one. Using this table, the parser evaluates all match transitions ($i \to i + k$) and literal-run transitions ($i \to i + c$ for $c \in 1..16$) to update the minimum bit cost to reach each subsequent position. Once the forward pass reaches N, the parser backtracks from N to 0 along the cheapest transitions to emit the final tokens.

However, optimizing purely for size often creates deep chains of back-references, where one match points to an earlier match, which points to another match, and so on. Decoding a single byte can then require following dozens of match hops before finally reaching a literal, slowing down random reads even with checkpoints.

To shorten reference chains without sacrificing our size-first design, we add optional *chain flattening* (`f1`) as a post-processing pass on the host. If a match token X falls completely inside an earlier match token Y that points to Z, chain flattening rewires X to point directly to Z (as long as the combined distance fits within the 16-bit limit), skipping Y entirely. Comparing `blaris_o2_a0_s8_f0` against `f1` (2-byte stream offsets, 2^0 = 1-byte alignment, 2^8 = 256-byte checkpoint spread, flattening off vs. on; Table 1), flattening cuts average reference jumps by 14%–51% across 17 of the 21 corpus files at a 0.5%–2.2% compression ratio cost (for example, halving average jumps on `zcu102_arm.dtb` from 45.6 to 22.4 at a 1.9% ratio cost). The remaining 4 files are incompressible (`cp.html.gz`, 0%)

Table 1: Impact of Chain Flattening (`o2_a0_s8_f0` vs. `f1`) on Reference Jumps (9 Representative Files; Full 21-File Table in Repository)

	Unflattened (`f0`)			Flattened (`f1`)			
Dataset	**Ratio**	**Avg**	**Max**	**ΔRatio**	**Avg**	**Max**	**ΔAvg**
`FreeRTOSConfig.h`	35.5%	13.7	143	+1.6%	11.2	140	−18.2%
`ch32v003_boot.bin`	95.4%	0.6	7	+0.5%	0.6	7	−14.1%
`fvp_base_gicv3.dtb`	33.5%	12.9	53	+1.5%	7.4	29	−42.5%
`grammar.lsp`	43.8%	6.2	23	+1.1%	4.8	19	−21.9%
`hifive-unleashed.dts`	51.3%	3.7	19	+1.0%	2.6	17	−30.2%
`rv32i_add.bin`	18.5%	3421.4	7699	+0.1%	3421.3	7696	0.0%
`speed_7578.csv`	23.3%	71.3	254	+1.1%	50.1	175	−29.8%
`zcu102_arm.dtb`	25.9%	45.6	226	+1.9%	22.4	81	−50.8%
`zynq7000s.dtb`	34.5%	16.9	62	+1.5%	12.8	49	−23.9%

or firmware binaries dominated by short local matches (`ch32v30x_blink.bin`, −0.6%) or multi-kilobyte zero-padding (`rv32i_add.bin` and `rv32i_fence_i.bin`, 0%). On the two zero-padded binaries, the optimal parser encodes zero blocks as self-overlapping run-length matches (`length=16, distance=1`), causing the decoder to step backward by $d = 1$ byte per hop across the entire zero run and preventing whole-token chain flattening.

Benchmarking Setup

We evaluated `blaris` against `heatshrink` and `tamp` across a 21-file corpus covering microcontroller firmware binaries, C source code, web assets (HTML/JSON), compiled Devicetree blobs (`.dtb`), and sensor CSV logs. All benchmarks were compiled with Clang (`-Os`) for 32-bit RISC-V (RV32IMC) and run in `rv32emu` [7] at 1 cycle per instruction with zero-wait-state memory.

For each file and codec configuration, we tested 9 slice sizes from 2^0 (1 B) to 2^8 (256 B). At each size, we decompressed 1,000 pseudorandom unaligned offsets and measured execution cycles via the RISC-V `rdcycle64` counter. We measured exact peak stack usage by filling the stack with `0xA5` canary bytes before execution and scanning for the deepest modified byte afterward.

All decompressors were wrapped in a common random-access API (`init`, `slice`, and `reset`). Because `heatshrink` and `tamp` only decode sequentially, we added a block-indexing wrapper that splits files into independently compressed blocks (2^4 to 2^{14} B) with a header offset table, allowing them to seek directly to the target block. Table 2 summarizes total Flash (`.text` + `.rodata`) and RAM (peak stack + `.bss` + `.data`) across all 206 tested configurations (108 `blaris`, 74 `heatshrink`, 24 `tamp`). Because `blaris` targets trusted build-time payloads, it omits runtime bounds and error checks to keep code size minimal.

Finally, to check whether SPI Flash wait states on real hardware disproportionately penalize `blaris`'s backward token walking, we tested a representative balanced configuration of each codec across the corpus on a physical ESP32-C3FN4 microcontroller in direct-boot XIP mode, mapping both decompressor code and the compressed payload to external quad-SPI Flash without copying either to SRAM.

Table 2: Decompressor Memory Footprints (RV32IMC, Including Adapter)

Codec	**Flash**	**Stack**	**Static RAM**	**Total RAM**
`blaris`	548–784 B	76 B	28–36 B	104–112 B
`heatshrink`	1,300–1,464 B	140–144 B	66–16,450 B	206–16,594 B
`tamp`	2,238–3,238 B	228–248 B	292–16,424 B	520–16,672 B

Results

Figure 1 shows the Pareto frontier of decompression speed versus compression ratio at slice sizes of 1 B, 32 B, and 256 B on a representative Devicetree file, `zynq7000s.dtb` (15.2 KiB).

On 1 B random reads (Fig. 1a), `blaris` holds 12 of the 14 Pareto-optimal points and runs over two orders of magnitude faster than ratio-matched `heatshrink` and `tamp` configurations. Because smaller independent blocks degrade compression ratio, matching `blaris`'s ∼ 36% ratio requires configuring `heatshrink` and `tamp` with a 1 KiB window (`w10`) and 16 KiB blocks (`b14`). At this ratio on `zynq7000s.dtb`, `blaris_o2_a0_s8_f1` takes 2,173 cycles/byte (0.46 B/kcyc) at a 35.99% ratio using 112 B of RAM, outperforming `heatshrink_b14_i16_w10_l6` (562× slower, 35.63% ratio, 1,214 B RAM) and `tamp_b14_w10_e0` (685× slower, 35.54% ratio, 1,288 B RAM). Similarly, on `FreeRTOSConfig.h` (35.7 KiB), `blaris` (2,311 cycles/byte, 37.10% ratio) achieves both a better ratio and 598× and 641× higher speed than the same `heatshrink` (43.83%) and `tamp` (41.31%) configurations.

At 32 B slices (Fig. 1b), `blaris` still holds 12 of the 16 sub-50%-ratio Pareto points (reaching 1.92 B/kcyc) while using only 104–112 B of total RAM. High-window `tamp` and `heatshrink` configurations join the faster end of the frontier, but require 5×–40× more RAM (520 B to > 4 KiB) for `tamp` and 2×–4× more (206–462 B) for `heatshrink`.

At 256 B slices (Fig. 1c), block-streaming codecs overtake `blaris`. Because `blaris` resolves matches without caching prior output in RAM, its cost grows linearly with slice length, whereas streaming decoders pay a one-time block-seek cost and then emit bytes cheaply. This places the crossover around 64–128 B slices: below this size, `blaris` offers superior speed and RAM efficiency; above it, streaming codecs are faster when sufficient RAM is available.

Comparing `rv32emu` against physical cached-XIP ESP32-C3FN4 silicon on one balanced configuration per codec (`blaris_o2_a0_s8_f1` across 19 files, excluding the two zero-padded `rv32i_add.bin` and `rv32i_fence_i.bin` binaries due to their $\mathcal{O}(N)$ RLE jump depth, and `heatshrink_b10_i16_w8_l4` and `tamp_b10_w8_e1` across all 21 files) shows that SPI Flash wait states do not disproportionately penalize `blaris` relative to the baselines. Overall hardware cycle counts increase by 29.3% for `blaris`, 33.5% for `heatshrink`, and 23.6% for `tamp`. On 1 B reads, `blaris` sees a 1.51× hardware overhead, likely because cold SPI Flash cache-line fills make up a larger share of its short execution time; for slices ≥ 4 B, the overhead stabilizes at 1.30×–1.33× (1.30×–1.31× for ≥ 8 B).

Conclusion

We presented `blaris`, a windowless LZSS decompressor designed for deeply constrained microcontrollers where RAM budgets are too tight for `heatshrink` or `tamp`. By resolving back-references directly in the compressed payload via bidirectional streams and compact power-of-two checkpoints, `blaris` achieves compression ratios comparable to high-window `heatshrink` and `tamp` configurations (which require > 1.2 KiB of RAM) while using only 548–784 B of Flash and 104–112 B of total RAM. For random access to small slices, `blaris` is up to orders of magnitude faster than ratio-matched, block-indexed streaming decoders, though streaming codecs become faster on larger sequential reads (≥ 128 B) when speed is the priority and sufficient RAM is available.

The complete `blaris` implementation, host compressors, and reproducible benchmark suite are available as open-source software at `https://github.com/lis05/blaris`.

References

[1] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, May 1977, doi: 10.1109/TIT.1977.1055714.

[2] J. A. Storer and T. G. Szymanski, "Data compression via textual substitution," *J. ACM*, vol. 29, no. 4, pp. 928–951, Oct. 1982.

[3] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," RFC 8878, IETF, Feb. 2021.

[4] S. Vokes, "heatshrink: data compression library for embedded/real-time systems," 2013. [Online]. Available: `https://github.com/atomicobject/heatshrink`

[5] B. Pugh, "Tamp: A lightweight, deflate-inspired compression library for microcontrollers," 2023. [Online]. Available: `https://github.com/BrianPugh/tamp`

[6] P. Sokolovsky, "uzlib: Deflate/Zlib/Gzip decompression library for embedded systems," 2014. [Online]. Available: `https://github.com/pfalcon/uzlib`

[7] Y.-F. Chen, M.-H. Chen, C.-C. Huang, and C.-H. Tu, "Accelerate RISC-V instruction set simulation by tiered JIT compilation," in *Proc. 16th ACM SIGPLAN Int. Workshop Virtual Mach. Intermed. Lang. (VMIL)*, 2024, pp. 12–22.

[8] S. Kreft and G. Navarro, "LZ77-like compression with fast random access," in *Proc. Data Compression Conf. (DCC)*, 2010, pp. 239–248, doi: 10.1109/DCC.2010.30.

[9] D. Kempa and B. Saha, "An upper bound and linear-space queries on the LZ-End parsing," in *Proc. ACM-SIAM Symp. Discrete Algorithms (SODA)*, 2022, pp. 2847–2866, doi: 10.1137/1.9781611977073.111.

[10] G. Navarro, "Indexing highly repetitive string collections, part II: Compressed indexes," *ACM Comput. Surv.*, vol. 54, no. 2, Art. no. 26, pp. 1–32, Mar. 2021, doi: 10.1145/3432999.

[11] I. Lukianov, "lzmpo: LZ77 multiple prefixes optimal compressor," 2026. [Online]. Available: `https://github.com/lis05/lzmpo`

[12] I. Pavlov, "7-Zip and LZMA SDK," 1999. [Online]. Available: `https://www.7-zip.org/sdk.html`

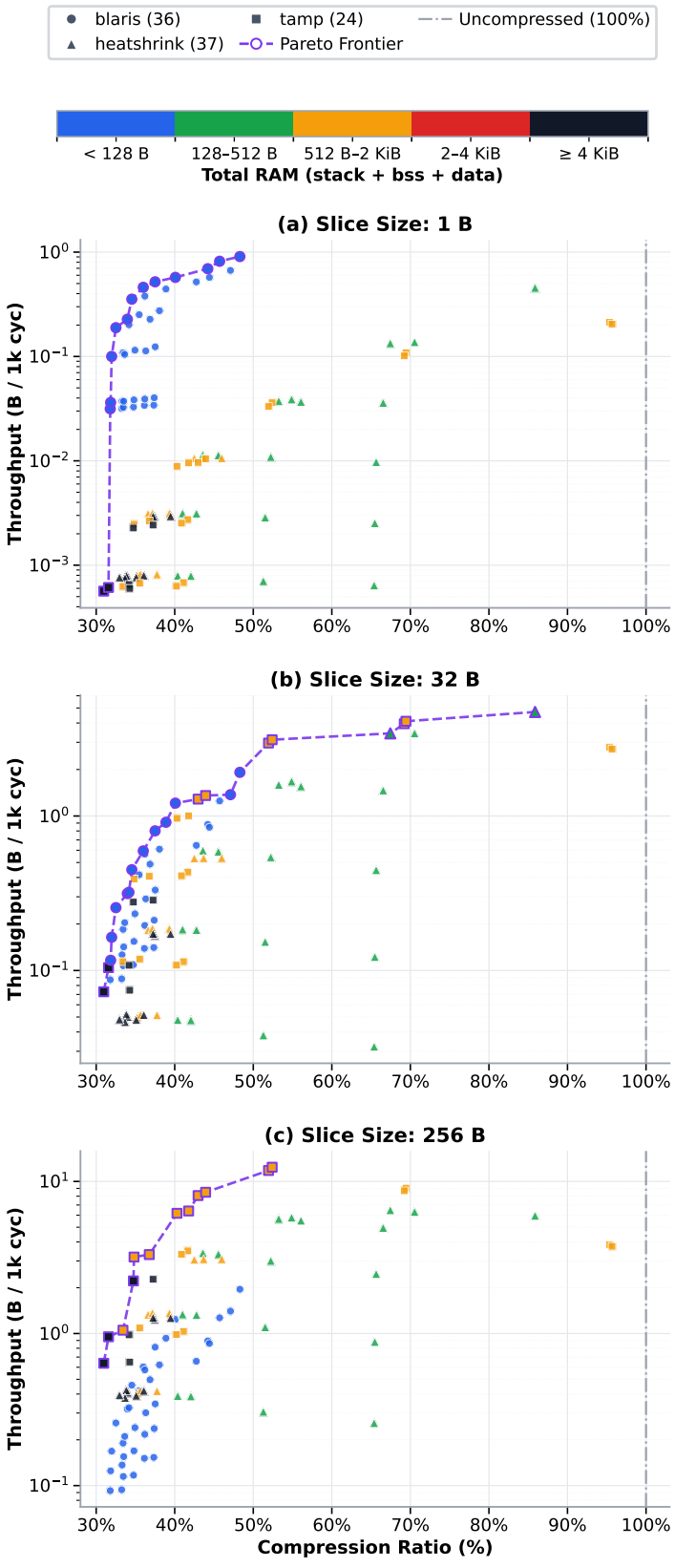


Figure 1: Decompression throughput vs. compression ratio on `zynq7000s.dtb` (15,583 B) at slice sizes of (a) 1 B, (b) 32 B, and (c) 256 B across 97 configurations (`blaris_o3`/`o4` and `heatshrink_i32` omitted for clarity).